

Compiler Design Lab Programs

6. Implement a C program to eliminate left recursion.

Code:

```
#include <stdio.h>

#include <string.h>

int main() {
    char nonTerminal;
    char alpha[50], beta[50];
    printf("Enter the non-terminal (A): ");
    scanf(" %c", &nonTerminal);
    printf("Enter alpha (recursive part after A -> Aα): ");
    scanf("%s", alpha);
    printf("Enter beta (non-recursive part after A -> β): ");
    scanf("%s", beta);
    printf("\nGrammar after eliminating left recursion:\n");
    char newNonTerminal = nonTerminal + 39; // Using ASCII to create A' as a simple trick
    printf("%c -> %s%c\n", nonTerminal, beta, newNonTerminal);
    printf("%c -> %s%c | ε\n", newNonTerminal, alpha, newNonTerminal);
    return 0;
}
```

Output:

Enter the non-terminal (A): A

Enter alpha (recursive part after A -> Aα): a

Enter beta (non-recursive part after A -> β): b

Grammar after eliminating left recursion:

A -> bh

h -> ah | ε

Output Screenshot:

```
A
a
b

Output:

Enter the non-terminal (A):
Enter alpha (recursive part after A -> A $\alpha$ ):
Enter beta (non-recursive part after A ->  $\beta$ ):

Grammar after eliminating left recursion:
A -> bh
h -> ah |  $\epsilon$ 
```

7. Implement a C program to eliminate left factoring.

Code:

```
#include <stdio.h>

#include <string.h>

int main() {

    char nonTerminal;

    char commonPrefix[50], beta1[50], beta2[50];

    printf("Enter the non-terminal (A): ");

    scanf(" %c", &nonTerminal);

    printf("Enter the common prefix ( $\alpha$ ): ");

    scanf("%s", commonPrefix);

    printf("Enter first suffix ( $\beta_1$ ): ");

    scanf("%s", beta1);

    printf("Enter second suffix ( $\beta_2$ ): ");

    scanf("%s", beta2);

    char newNonTerminal = nonTerminal + 39; // Using ASCII to create A'

    printf("\nGrammar after left factoring:\n");

    printf("%c -> %s%c\n", nonTerminal, commonPrefix, newNonTerminal);

    printf("%c -> %s | %s\n", newNonTerminal, beta1, beta2);

    return 0;

}
```

Output:

Enter the non-terminal (A): A

Enter the common prefix (α): a

Enter first suffix (β_1): b

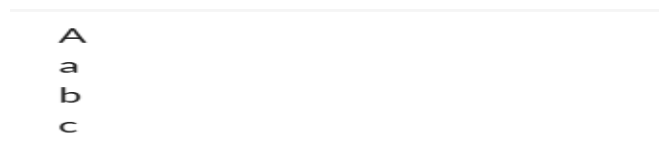
Enter second suffix (β_2): c

Grammar after left factoring:

A $\rightarrow$ ah

h $\rightarrow$ b | c

Output Screenshot:



```
A
a
b
c
```

Output:

```
Enter the non-terminal (A):
Enter the common prefix ( $\alpha$ ):
Enter first suffix ( $\beta_1$ ):
Enter second suffix ( $\beta_2$ ):
```

```
Grammar after left factoring:
A  $\rightarrow$  ah
h  $\rightarrow$  b | c
```

8. Implement a C program to perform symbol table operations.

Code:

```
#include <stdio.h>

#include <string.h>

struct Symbol {
    char name[20];
    char type[10];
    int value;
};

int main() {
    struct Symbol table[50];
    int n, i;
    printf("Enter number of symbols: ");
    scanf("%d", &n);
    for (i = 0; i < n; i++) {
        printf("\nEnter symbol %d name: ", i+1);
```

```

scanf("%s", table[i].name);

printf("Enter symbol %d type: ", i+1);
scanf("%s", table[i].type);
printf("Enter symbol %d value: ", i+1);
scanf("%d", &table[i].value);
}

printf("\nSymbol Table:\n");
printf("Name\tType\tValue\n");
printf("-----\n");
for (i = 0; i < n; i++) {
    printf("%s\t%s\t%d\n", table[i].name, table[i].type, table[i].value);
}

return 0;
}

```

Output:

Enter number of symbols: 2

Enter symbol 1 name: x

Enter symbol 1 type: int

Enter symbol 1 value: 10

Enter symbol 2 name: y

Enter symbol 2 type: int

Enter symbol 2 value: 20

Symbol Table:

Name	Type	Value

x	int	10
y	float	20

Output Screenshot:

```
2
x
int
10
y
float
20
```

Output:

```
Enter number of symbols:
Enter symbol 1 name: Enter symbol 1 type: Enter symbol 1 value:
Enter symbol 2 name: Enter symbol 2 type: Enter symbol 2 value:
Symbol Table:
Name      Type      Value
-----
x         int       10
y         float     20
```